When does A* save work?

T01: structure, heuristic and the cost of guidance

Elnur Haqqverdiyev Rajab Iskandarli

UFAZ, M1 Data Science and AI
Advanced Data Structures and Algorithms, Fall 2026

Defence

The question

Dijkstra expands every vertex closer than the target.
A* adds a heuristic $h(v)$ and expands fewer vertices.

But guidance is not free:

- one heuristic evaluation per generated vertex;
- larger heap keys / more memory traffic;
- a weak heuristic can save nothing.

Research question

How do graph structure and heuristic strength change

- expanded vertices and
- wall-clock runtime

of A* versus Dijkstra, on unit-cost grids and a real road graph?

The answer must separate **expansions** from **time**.

Data: synthetic controls and a real graph

Grids — 4-neighbour, unit cost, 128–1024 square

- open
- random 20% obstacles
- clustered 30% obstacles
- rooms (walls with doors)
- perfect maze

Largest 4-connected component is used. Blocked cells are isolated vertices.

Real graph: SNAP `roadNet-CA`

- California road network
- 1,957,027 vertices after symmetrisation and largest component
- 2,760,388 undirected edges (5,520,776 CSR arcs)
- unit hop weight: SNAP has no lengths/travel times

One sparse, planar-like road graph; one limitation, not a sample of all real graphs.

experiment	query instances	timed runs
grids (20 graphs)	300	2,700
real graph	30	900
total	330	3,600

What we built: ADTs and complexity

Graph: CSR

`offsets[n+1]`, `targets[m]`, `weights[m]`
memory $O(n+m)$; neighbour iteration $O(\deg u)$

Binary heap

push/pop $O(\log n)$ worst case; top $O(1)$
lazy deletion: no decrease-key; stale entries are detected on pop
ties: `node_id` vs `larger_g`

One search loop

Dijkstra = same loop with $h \equiv 0$
A* = $f = g + h(v)$; stop when target is settled
consistent heuristic $\Rightarrow$ no reopening
scratch arrays reset in $O(1)$ by generation stamp

Heuristics

grids: Manhattan (admissible + consistent)
road: ALT $\max_L |d(L,v) - d(L,t)|$
16 landmarks by farthest-point sampling; main $K = 8$

Correctness first

Tests before any timing

- heap order vs reference priority queue
- hand-computed 5x5 wall grid: distance 8
- hand-computed weighted graph vs Floyd–Warshall
- all five grid families vs independent BFS
- ALT admissibility and consistency for every free vertex
- path validation: contiguous edges, weights sum to distance

Oracle and audit

Every benchmark query is checked against BFS distance.

3,600 / 3,600 runs: `status=ok`, distance matches oracle, zero timeouts.

Expansions deterministic across repeats; median time CV $\approx$ 7–10%.

counter	definition (short)
expanded	popped, not stale, not closed; target included
pushed	source + every successful distance decrease
stale_popped	old heap entry or already-closed vertex
time_ns	search call only; inputs outside timer

Frozen protocol

- paired Dijkstra/A* on exactly the same query
- 15 grid queries/graph, 30 real queries
- 3 grid repeats, 5 real repeats
- one untimed warm-up per treatment
- report median per query; keep raw repeats
- bootstrap 95% CI + paired Wilcoxon
- 30 s cap per search; 4 GiB process cap
- one CPU via `taskset`

Query bands

BFS distance from a random source:

- short: 8–25% of eccentricity
- medium: 40–60%
- long: 80–97%

Selection uses only BFS, never Dijkstra/A*.

Freeze

Predictions in `HYPOTHESES.md`; protocol fixed before the main run in `PROTOCOL.md`.

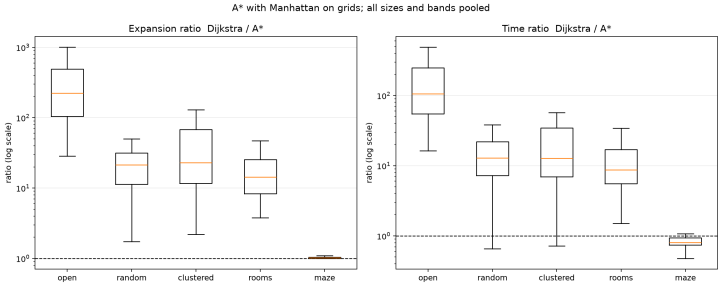
Predictions before the main run

	prediction	verdict
H1	open grid: A* saves many expansions and time	confirmed
H2	random obstacles: still saves, less than open	confirmed
H3	clustered/rooms: weaker than open; rooms weaker than random	confirmed
H4	maze: no expansion saving; A* slower	confirmed
H5	long queries benefit more than short	partly rejected
H6	ALT strong at K=8; K=1 may not help; diminishing at 16	partly rejected
H7	`larger_g` reduces median expansions on open	rejected as median claim

The protocol explicitly allowed negative and null results.

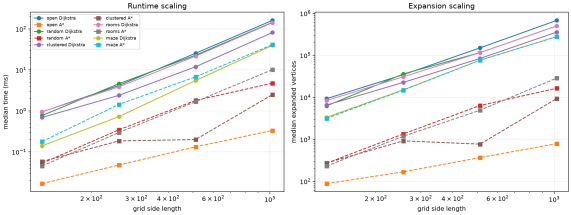
Main grid result

family	time ratio (median)	expansion ratio (median)	time saving (median)	A* faster queries
open	**105.7x**	**222.4x**	99.1%	100%
random 20%	12.8x	21.4x	92.2%	98%
clustered 30%	12.8x	22.9x	92.2%	98%
rooms	8.7x	14.3x	88.5%	100%
maze	**0.81x**	**1.02x**	**–24.0%**	17%



Ratio = Dijkstra / A*; values above 1 mean A* wins.

Structure matters more than distance



Open grid: A* follows one corridor; the advantage grows with size (390x time, 696x expansions at 1024).

Random/clustered: local detours cost some guidance power; still 8–13x time.

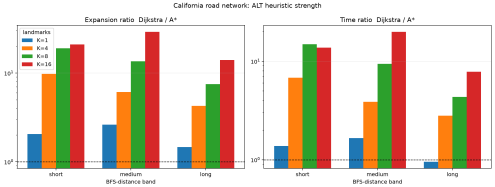
Rooms: long forced detours; weakest positive family (8.7x).

Maze: Manhattan cannot see the forced turns. A* is **24% slower**; expansions are essentially unchanged.

Real road network: ALT makes A* worthwhile

ALT	time ratio	exp. ratio	time saving	faster
K=1	1.29x	1.96x	22.5%	70%
K=4	4.75x	7.26x	78.9%	100%
K=8	**7.62x**	**11.54x**	86.8%	100%
K=16	**13.52x**	**22.41x**	92.6%	100%

K=1 time difference not significant (p=0.13); K=4/8/16 p<1e-8.
K=16 is 21% faster than K=8 on paired queries.

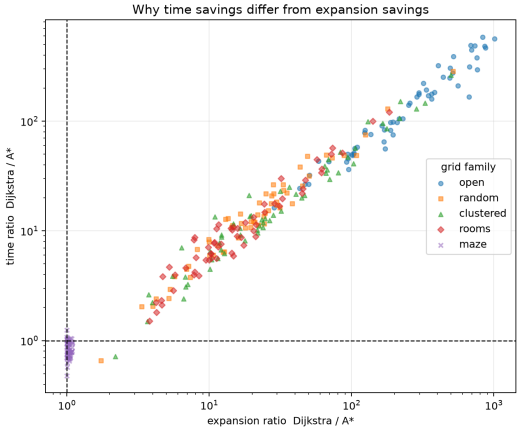


More landmarks keep tightening the bound; no plateau at 8 on this graph.

Why time saving is smaller than expansion saving

family	Dijkstra ns/exp	A* ns/exp	ratio
open	142	288	2.03x
random	157	254	1.62x
clustered	127	245	1.93x
rooms	146	270	1.85x
maze	65	79	1.21x

Real graph: Dijkstra 235 ns/exp; ALT K=8 373 ns (1.59x); K=16 394 ns (1.68x).

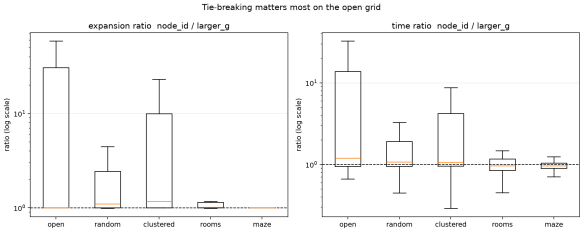


The rule: time is saved only when the expansion reduction pays for the more expensive expansion.

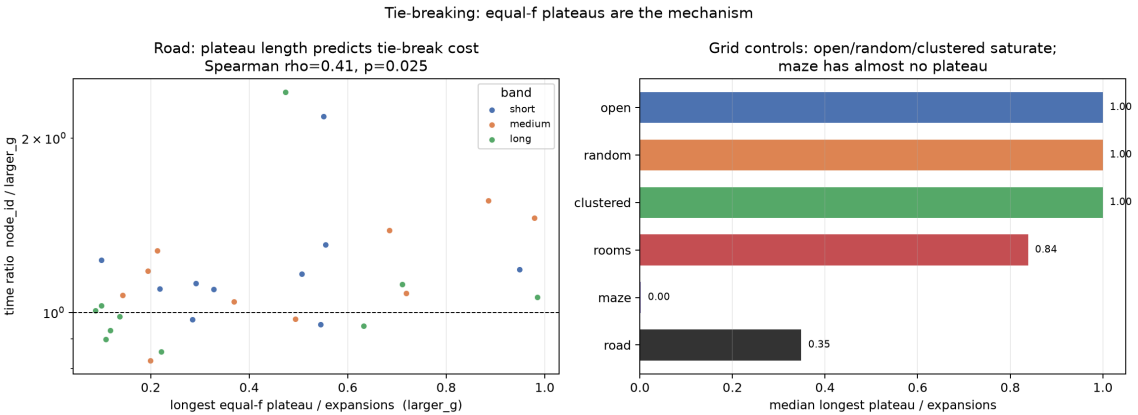
Tie-breaking: a tail-risk decision

family	median ratio	node_id worse queries	worst ratio
open	1.00	45%	285.7x
random	1.10	63%	51.5x
clustered	1.17	68%	92.6x
rooms	1.02	58%	17.8x
maze	1.00	2%	1.01x
road (ALT K=8)	1.14	90%	2.39x

Ratio = node_id expansions / larger_g expansions. On open, `larger_g` never expanded more; it removes the catastrophic tail. On the road graph, node_id is worse on 27/30 queries (expansion p= 4.4×10^{-6}).



Why tie-breaking matters: equal-f plateaus



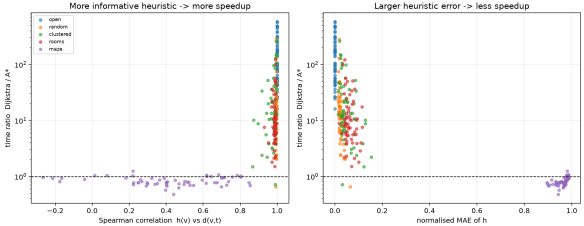
A plateau is a maximal run of settled vertices with the same f. The open grid is one saturated plateau (1.00), so the tie-break decides the route; the maze has almost none (0.002) and is neutral. On the road graph the plateau is intermediate (0.35) and predicts the penalty: expansion $\rho = 0.53$, $p = 0.003$; the high-plateau third pays 1.25x median time.

What makes a heuristic informative?

family	h corr.	norm. MAE	time ratio
open	1.00	0.00	105.7x
random	1.00	0.02	12.8x
clustered	0.99	0.04	12.8x
rooms	0.99	0.06	8.7x
maze	0.42	0.97	0.81x

Pooled Spearman: $+0.78$ for h-distance correlation, -0.81 for normalised MAE. A one-feature model has $R^2 = 0.55$.

Admissibility is not enough: the heuristic must be *informative* about the actual remaining distance.



Weighted A*: price of optimality

workload	ϵ	speed	cost
random	1.10	20.1x	1.018
random	1.25	38.2x	1.055
rooms	1.25	24.8x	1.031
clustered	1.50	26.5x	1.050
maze	50	1.10x	1.000
road	1.00	7.4x	1.000
road	1.10	8.8x	1.014
road, reopen	1.10	5.4x	1.011

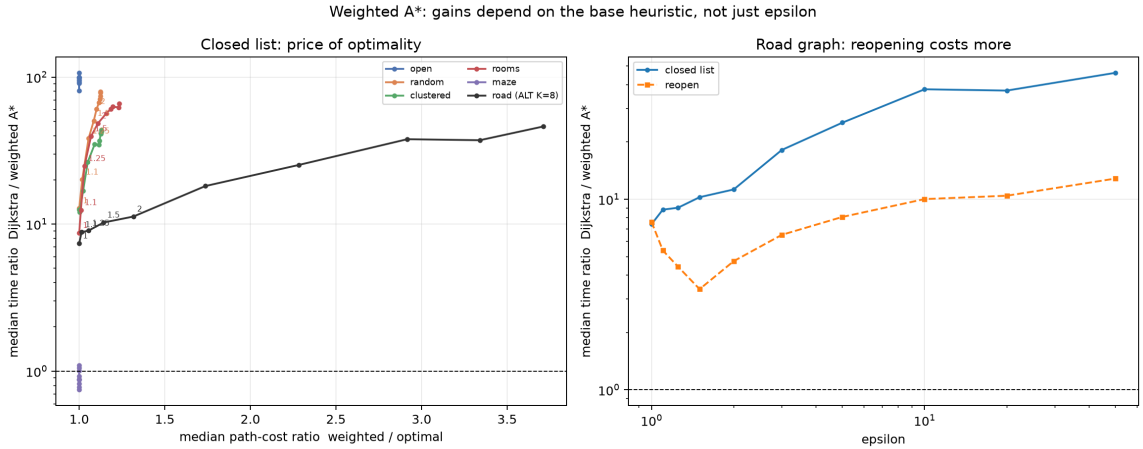
$f = g + \epsilon h$, closed list except the last row. All five grid families and the 30 road queries. Speed is Dijkstra/weighted A*. Maze and open grid are controls: their unique or exact paths keep cost at 1.00.

Weak heuristic: the first few percent of path cost buy a large amount of search. Random obstacles move from 12.8x to 20.1x for 1.8% median cost.

Strong heuristic: on the road graph there is little to gain. The closed list improves 7.4x to 8.8x at 1.4% cost; reopening falls to 5.4x because reheapifying settled vertices costs more than the shorter search saves.

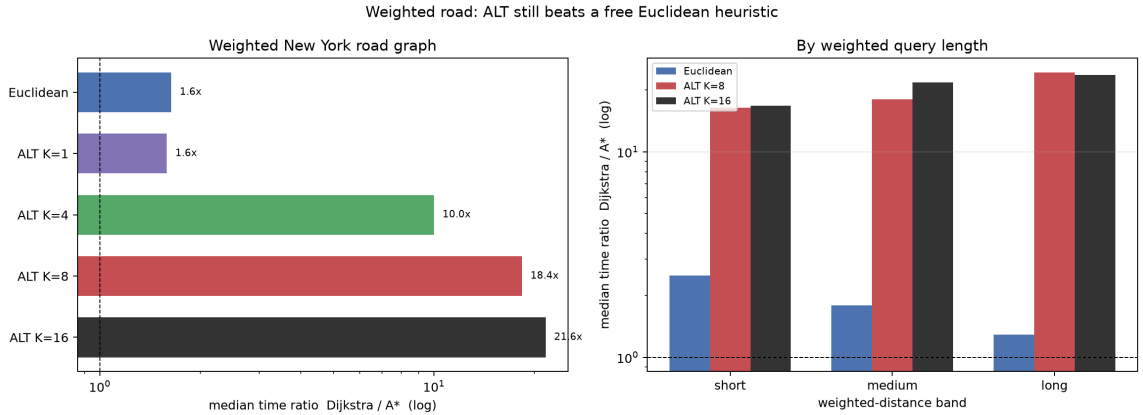
Rule: weight when the heuristic is weak, keep $\epsilon \approx 1.1$–1.25 if quality matters, and prefer a closed list.

Weighted A*: cost versus speed



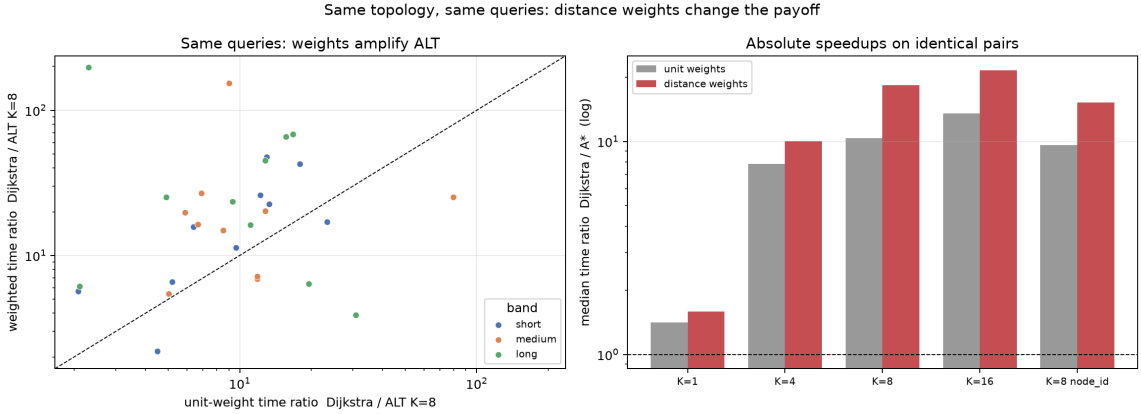
Left: closed-list frontier; points at cost 1.00 are the open grid and perfect maze, where there is no alternative path to trade. Right: on the road graph the closed list is faster at every $\epsilon > 1$, while reopening churns the heap for no useful gain.

Weighted road: distance, not hops



DIMACS New York, 264k nodes, distance weights and coordinates. Weighted Dijkstra oracle; all 30 queries optimal. ALT K=8 gives 18.4x median time and K=16 21.6x; the free Euclidean heuristic gives only 1.6x. Weighted edges also reduce the `node_id` expansion penalty to 1.08x.

Same topology: what did the weights change?

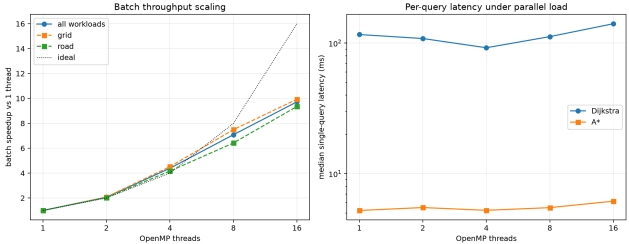


Identical 30 source-target pairs, only the edge weights differ. ALT K=8 rises from 10.4x (unit) to 18.4x (distance weights), while the `node_id` time penalty falls from 1.17x to 1.08x. Weighted A* with a closed list reaches 35.1x at 1.1% median cost.

Does A* still win with 16 threads?

threads	batch	speedup	efficiency
1	44.39 s	1.00x	1.00
2	21.48 s	2.07x	1.03
4	10.11 s	4.39x	1.10
8	6.27 s	7.08x	0.89
16	4.56 s	**9.73x**	0.61

Five 1024^2 grids + 30 road queries, 3 repeats. Each task runs Dijkstra and A* on the same query; thread-local scratch and heuristic. A* advantage remains: median Dijkstra/A* ratio 14.8x at one thread, 17.9x at 16. Maze stays below parity.



Throughput gain, not single-query speedup.

What the data rejected

- **H5:** long queries do not always benefit more. Open grid: long 156x vs short 83x time; random grid: long 11.5x vs short 14.4x.
- **H6 (diminishing returns):** K=16 is still better than K=8 on paired real queries (21% time, 25% expansions, p<1e-4).
- **H7 (median tie-break):** grid median is exactly 1.00 on the open grid, although 45% of queries are worse with `node_id`, up to 286x. On the road graph, node_id is worse on 27/30 queries (median 1.14x, worst 2.39x).
- **The headline negative result:** in the maze, A* is 24% slower even though its distances are correct.

Interpretation

A null or negative result is still a result: guidance only pays when it changes the explored region, not merely when it is admissible.

Robustness: host performance mode

workload	queries	Dijkstra speed-up (low/high mode)	A* speed-up (low/high mode)	ratio change (high/low)	95% CI
grid, sizes 128/512	150	2.24x	2.23x	1.011	0.994–1.028
road, ALT K=8	30	2.26x	2.23x	0.993	0.967–1.020

Matched subset

Same graphs, queries, seeds, bands and repeats as the frozen run. Expansions and distances are identical in both modes; only wall-clock time changes.

Result

The higher mode makes both algorithms ≈2.24x faster and leaves the A* ranking unchanged. The maze loss persists; only absolute nanoseconds are mode-specific.

No family changes significantly except open grid (ratio 0.954, $p = 0.058$), where A* runs in only 17–133 μs per query.

Threats to validity

- one real graph; road network, not a sample of all real graphs
- unit hop weights; no travel times in SNAP
- symmetrised roads; one-way streets not modelled
- synthetic grid families are controls, not real maps
- one machine, one heap implementation

- medians and IQRs, bootstrap CIs, paired Wilcoxon
- within-query time CV 7–10%
- all 3,600 runs match BFS; no timeouts
- raw CSV and seeds are kept
- protocol frozen before the main run

The constants are implementation- and machine-specific; the direction and the mechanism are the transferable result.

Final answer

When does A* save work?

When the heuristic is informative about the *actual* remaining path, and the reduction in expanded vertices is large enough to pay for the more expensive expansion.

- **Open grid:** yes, spectacularly (106x time, 222x expansions).
- **Random/clustered/rooms:** yes, but 9–13x time, not 100x.
- **Maze:** no; 24% slower, because Manhattan cannot see walls.
- **Road network:** yes; ALT K=8 7.6x, K=16 13.5x.
- **Weighted A*:** a closed list buys 20x at 1.8% cost on random obstacles; on the road graph reopening costs more than it saves.
- **Tie-break:** `larger_g` is the safe default; `node_id` is worse on 27/30 road queries.
- **Practical rule:** measure expansions and cost per expansion; do not assume the heuristic is free.

Reproduce

Build and test

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
./build/test_core
```

Full frozen run

```
./scripts/run_experiments.sh
```

Artifacts

`HYPOTHESES.md`, `PROTOCOL.md`, `RESULTS.md`,
`results/grid.csv`, `results/real.csv`, `results/analysis.md`, `figures/`

Questions?